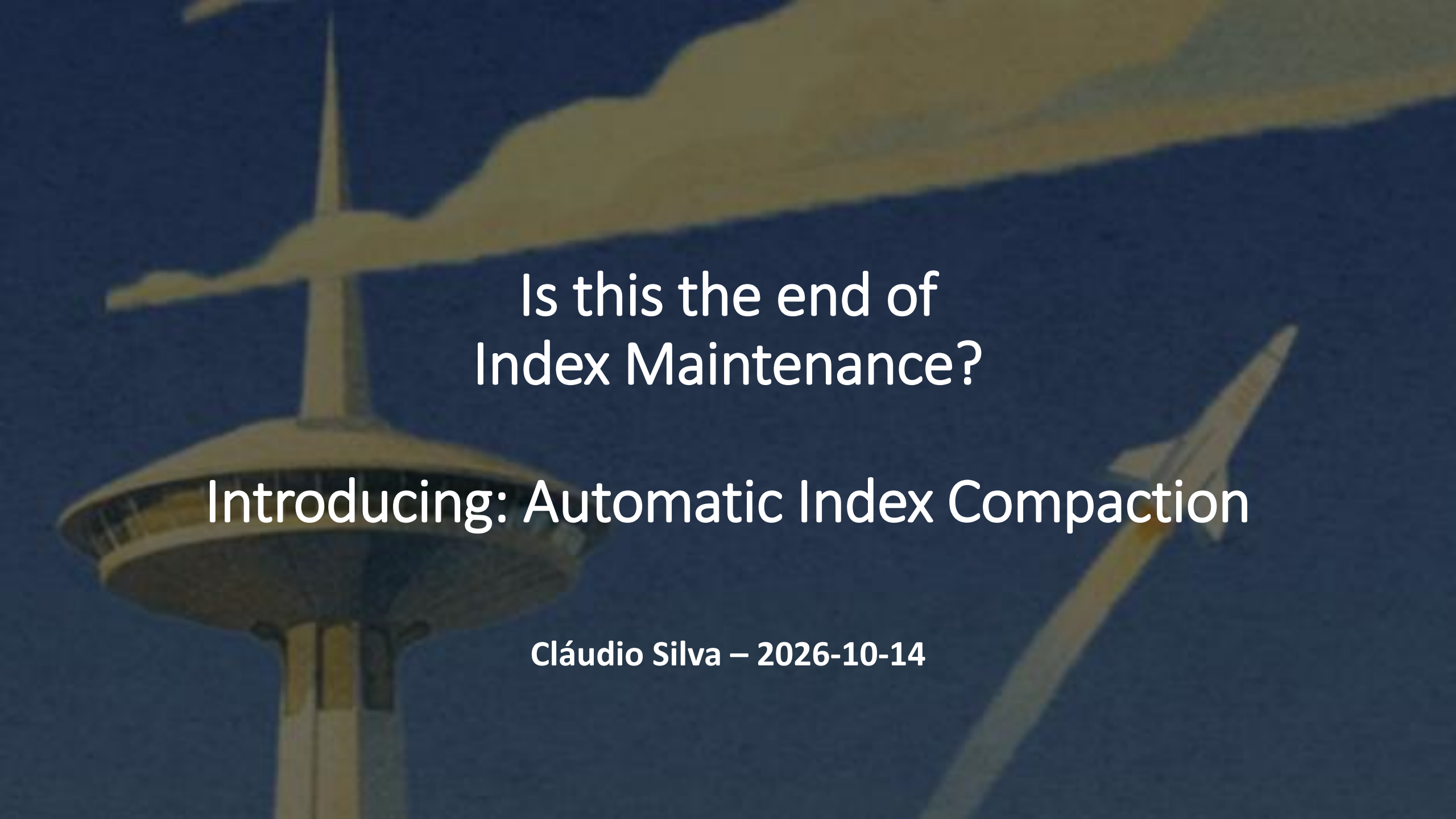


Is this the end of Index Maintenance?

Introducing: Automatic Index Compaction

Cláudio Silva – 2026-10-14

Thank you, partners

















































Is this the end of Index Maintenance?



Source: https://giphy.com/gifs/fallontonight-jimmy-fallon-anticipation-drumroll-CLCT8BXR1T2oyBYilM

Is this the end of Index Maintenance?



Not really!

Is this the end of Index Maintenance?

Not really!



Source: https://giphy.com/gifs/season-16-the-simpsons-16x7-l2JdSsEmnouFrNz7W

Is this the end of Index Maintenance?



Thanks for coming to my TED Talk



Source: https://giphy.com/gifs/TEDxUNebrija-ted-tedx-tedxunebrija-2UJvTeZySHq3LscGqy

Is this the end of Index Maintenance?



It’s a big step in the right direction

Is this the end of Index Maintenance?

It’s a big step in the right direction



Source: https://giphy.com/gifs/happy-excited-applause-BQAk13taTaKYw

Who am I?

Cláudio Silva

(He / Him)



What do I do?

masterminds group

- Performance Tuning
- Automation

data masterminds

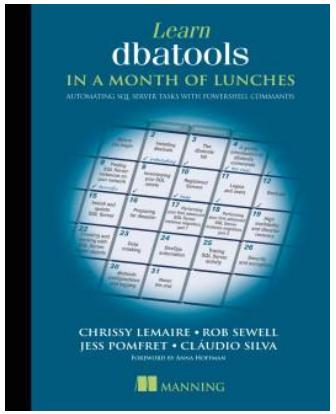
analytics masterminds

devops masterminds

Open-Source Contributor

- dbatools
- dbachecks

MVP Microsoft® Most Valuable Professional



claudioessilva

claudioessilva.eu

claudioessilva

A bit of history about index maintenance



- The suggested defaults of 5/30
- The spinning disks era
- The workload type
- Wasted space



Later...

- Faster disks
- SSDs appear
- The suggested defaults are questioned
- NVMe appear, too





Indexes fragmentation



- External fragmentation
  - Also known as “pages out of order”.
  - With newer hardware, it is less of/not a big problem

- Internal fragmentation
  - Also known as “page density”
  - Still has a performance penalty

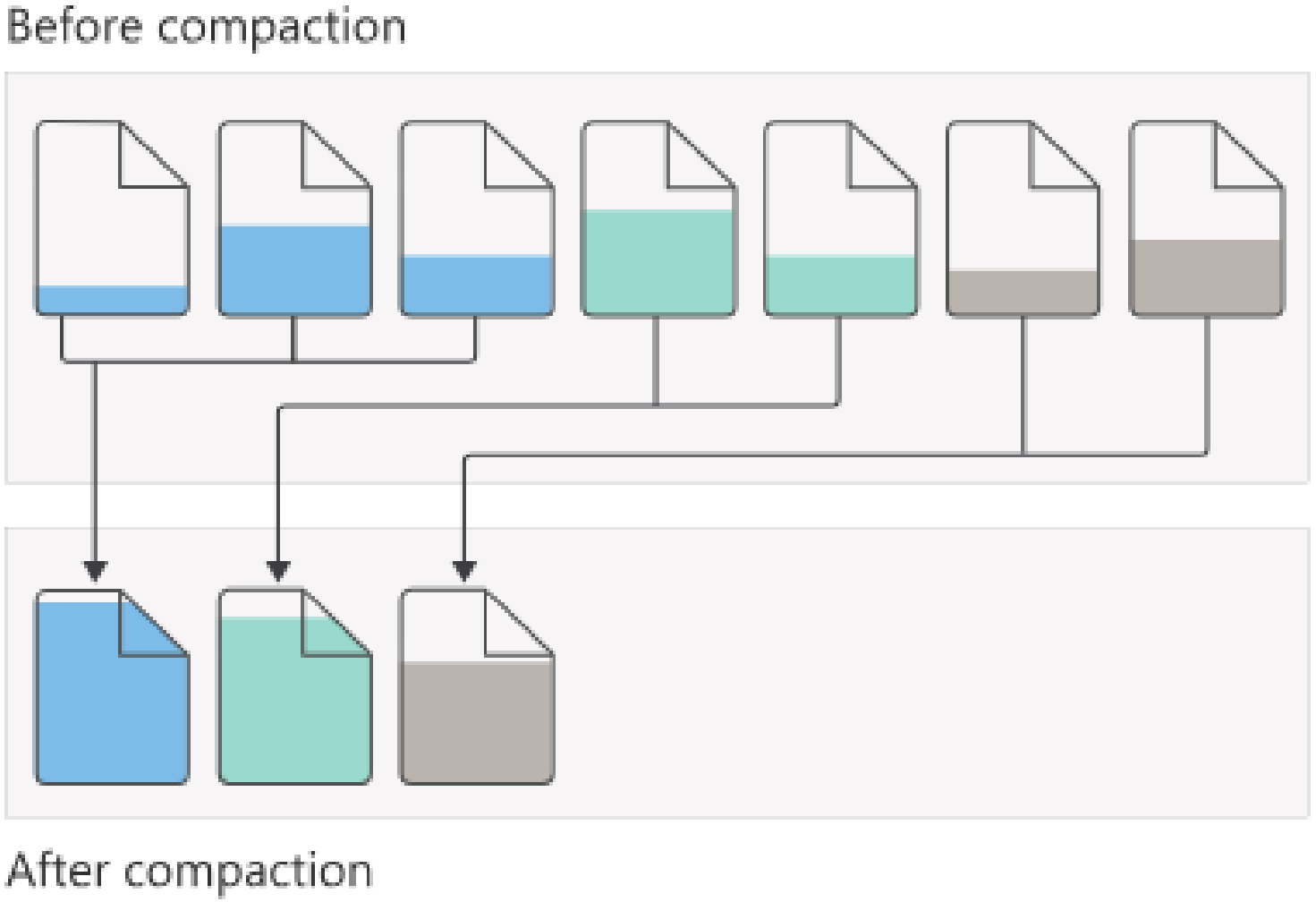
- Leads to more:
  - Logical reads (pages read)
  - Elapsed time
  - CPU time
  - More storage
  - Memory (buffer pool)
  - Backups (Bigger)

Current options for indexes defragmentation



- Reorganize
  - It physically reorders the leaf-level pages to match the logical order of the leaf nodes, left to right.
  - Compacts index pages to make page density equal to the fill factor of the index.
  - Always online
- Rebuild
  - Rebuilding an index drops and re-creates the index.
  - Depending on the type of index and the Database Engine version, a rebuild operation can be done offline or online.

AIC - How it works



Source: [Automatic Index Compaction - SQL Server | Microsoft Learn]

Don’t be surprised if the index size is bigger



Source: giphy.com

Don’t be surprised if the index size is bigger



Not AIC-related, but rather an artifact of versioning



Don’t be surprised if the index size is bigger



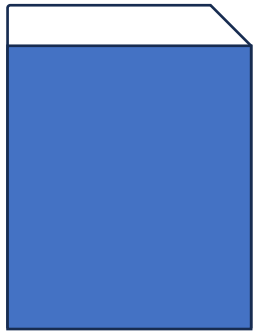
Let me explain it!

- When rows are inserted under ADR, each gets the 14-byte version pointer.
- When you delete rows under ADR, a row is not physically removed.
  - It is ghosted, and a 14-byte version pointer is added
    - This addition can cause page splits on densely packed pages
    - This can happen again while the DELETE keeps adding version pointers
  - Result? More pages, bigger index.

- Ghost cleanup
  - Removes deleted row versions.
  - Deallocates any empty pages within extents – but doesn’t purge them!

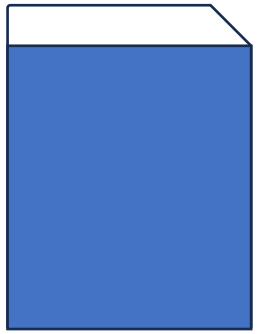
- AIC runs part of the version cleanup
  - It consolidates rows on remaining allocated pages and deallocates empty pages including purging them.
  - But doesn’t do anything with the empty extents

Don’t be surprised if the index size is bigger



Density ~99.9%

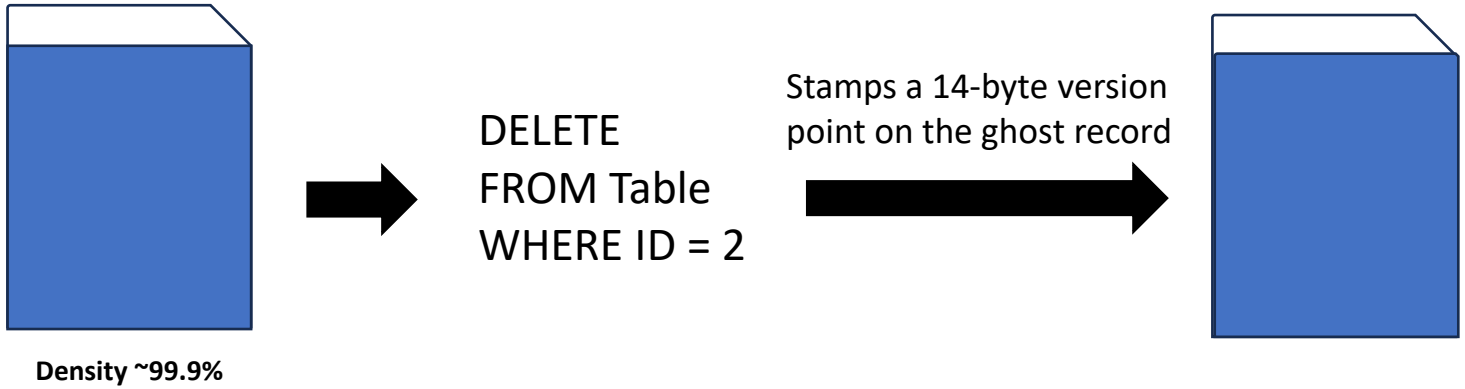
Don’t be surprised if the index size is bigger



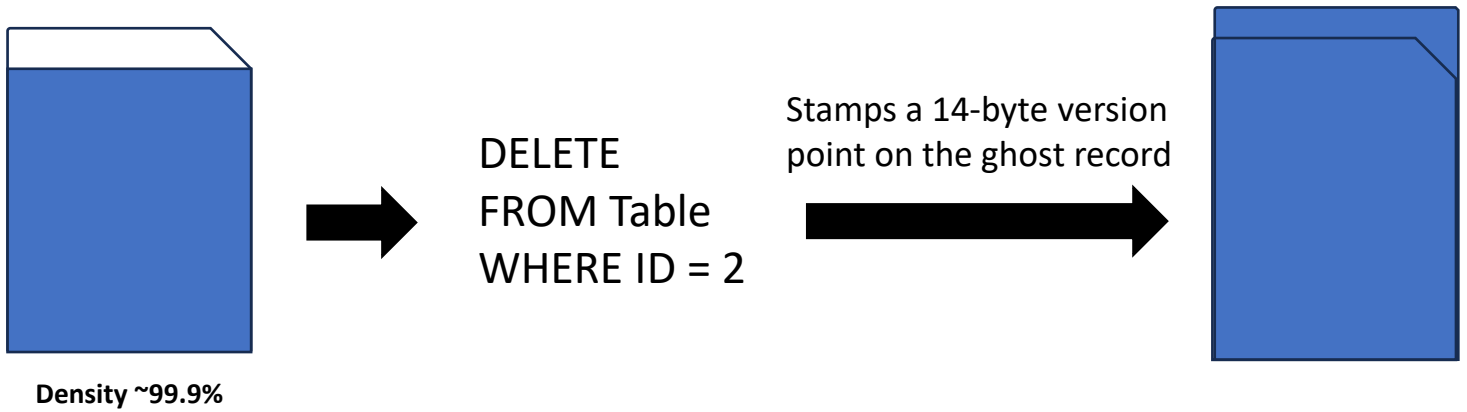
Density ~99.9%

DELETE
FROM Table
WHERE ID = 2

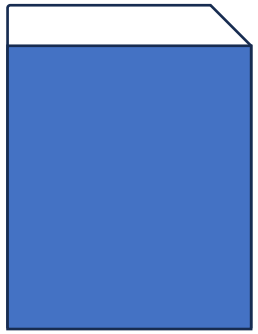
Don’t be surprised if the index size is bigger



Don’t be surprised if the index size is bigger



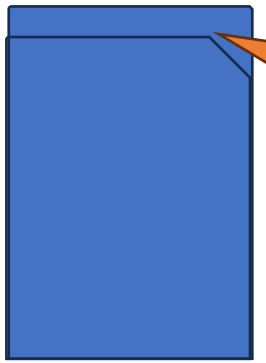
Don’t be surprised if the index size is bigger

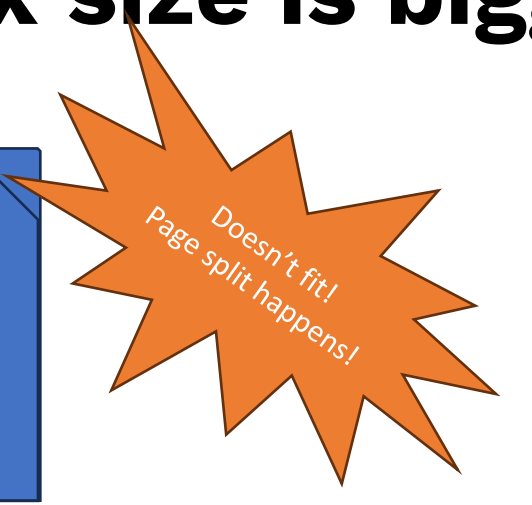


Density ~99.9%

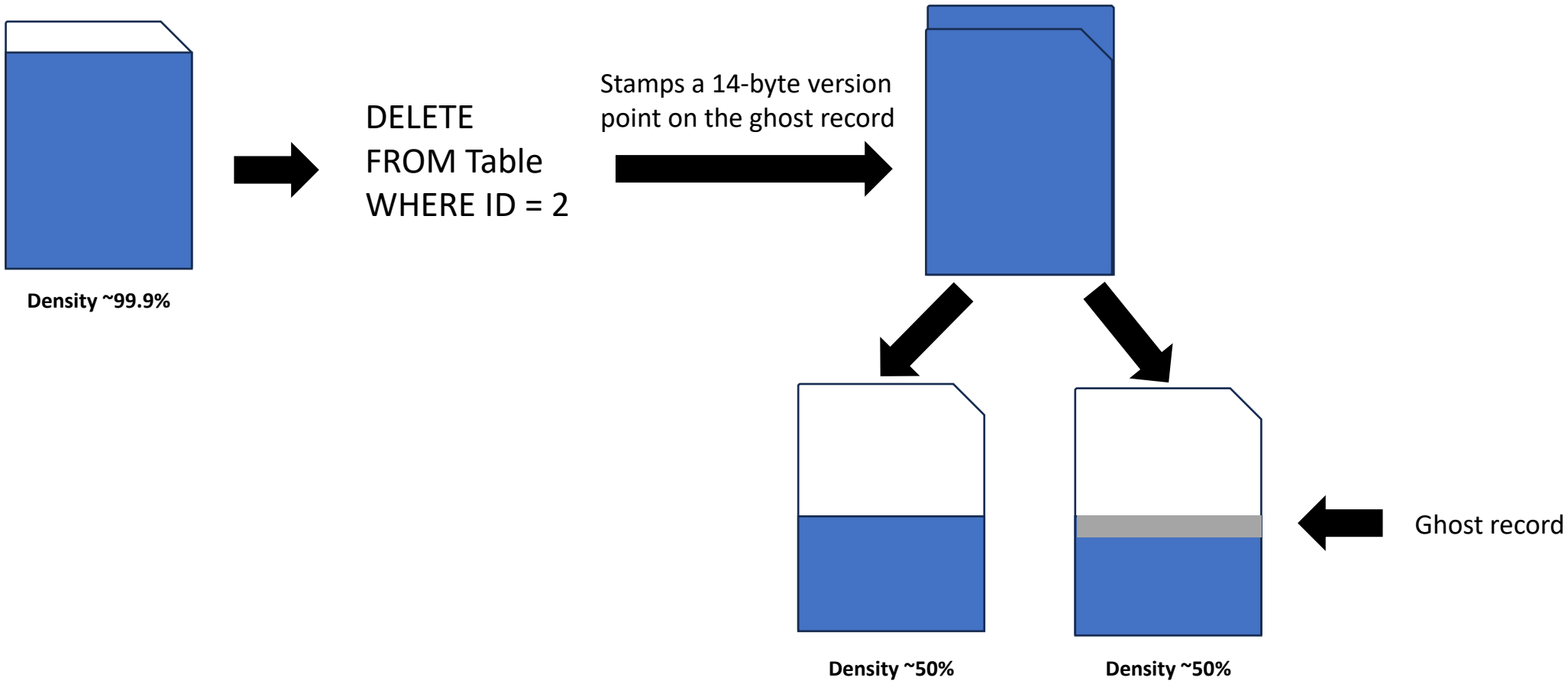
DELETE
FROM Table
WHERE ID = 2

Stamps a 14-byte version point on the ghost record

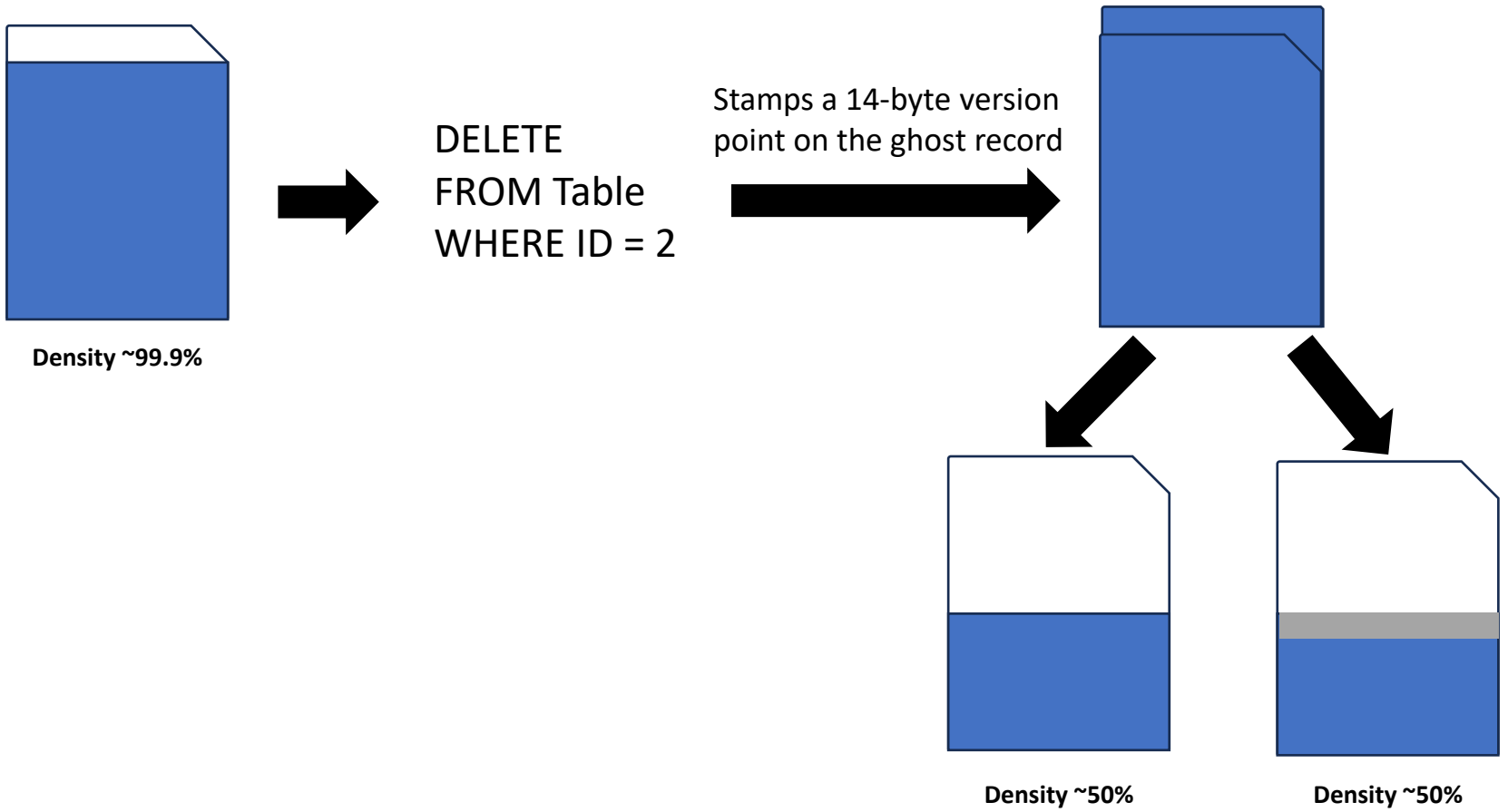




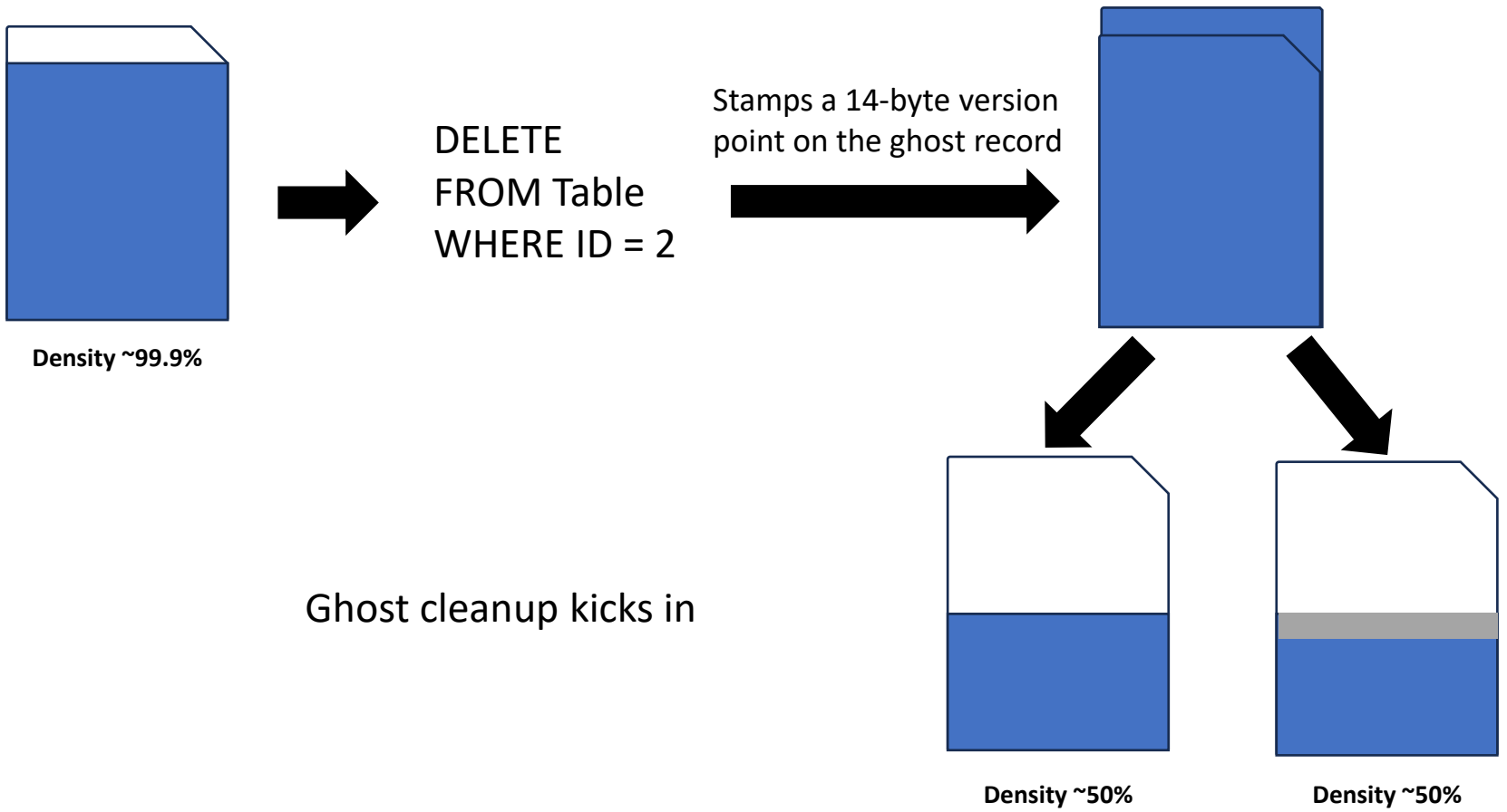
Don’t be surprised if the index size is bigger



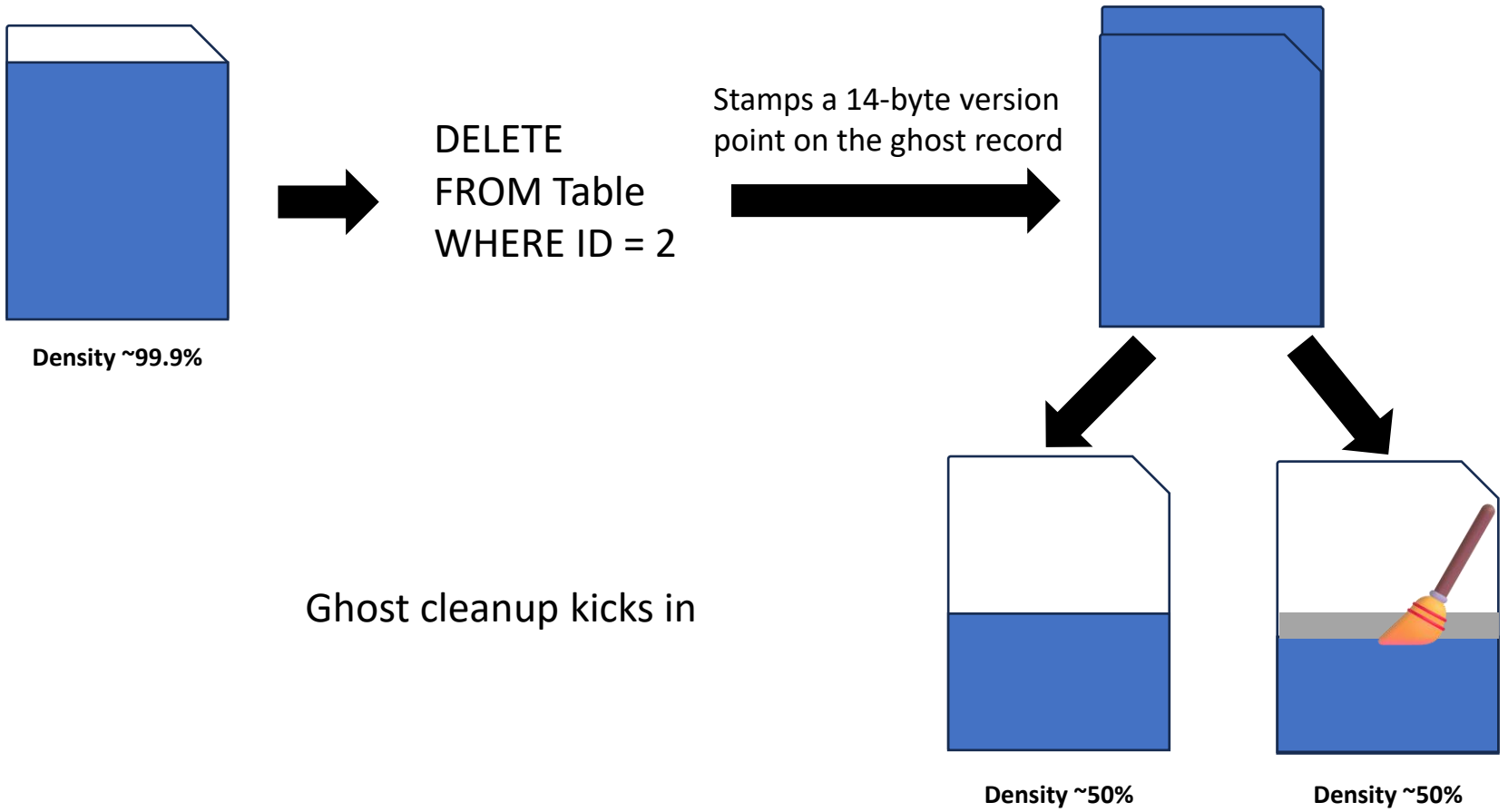
Don’t be surprised if the index size is bigger



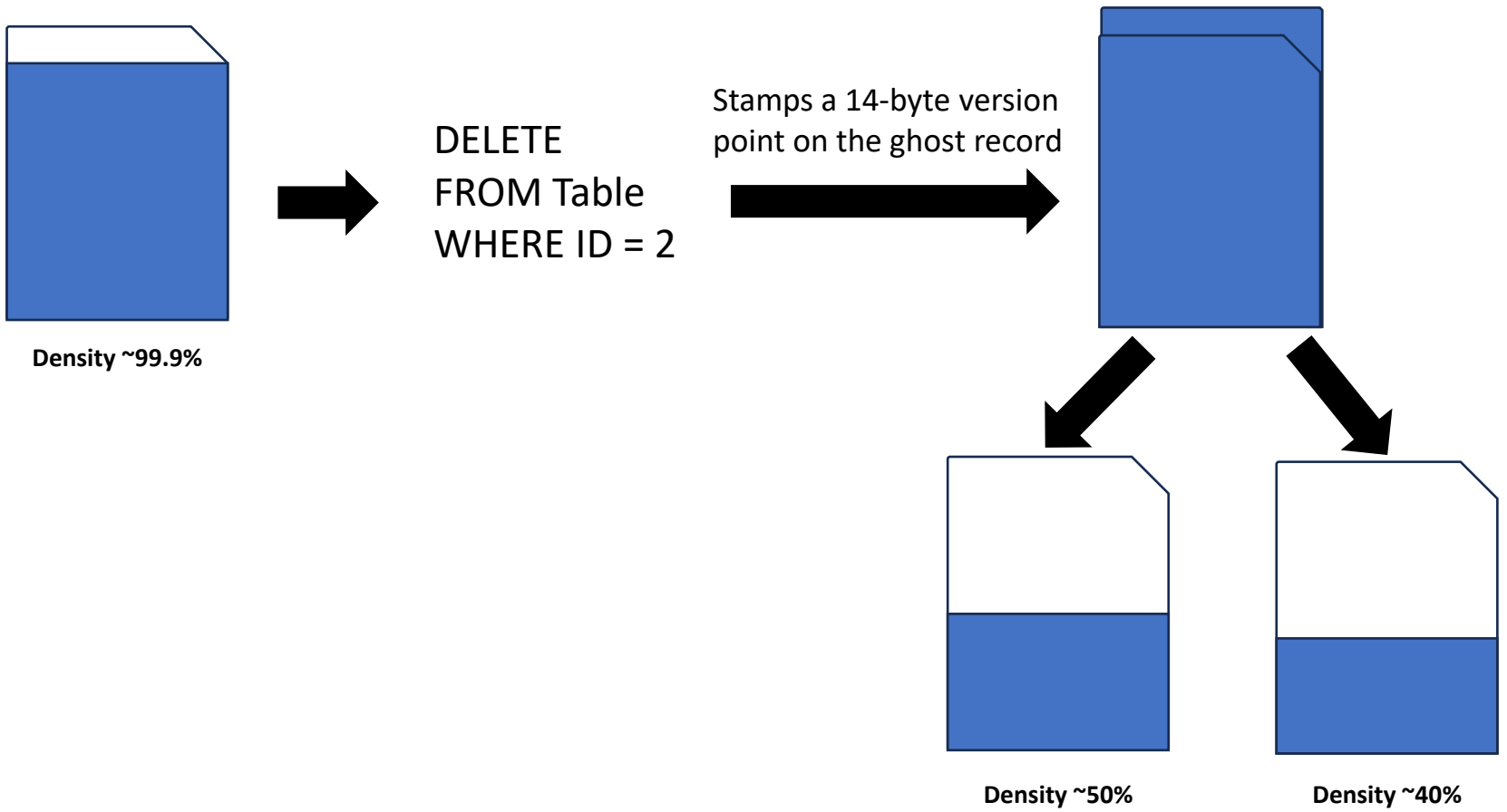
Don’t be surprised if the index size is bigger



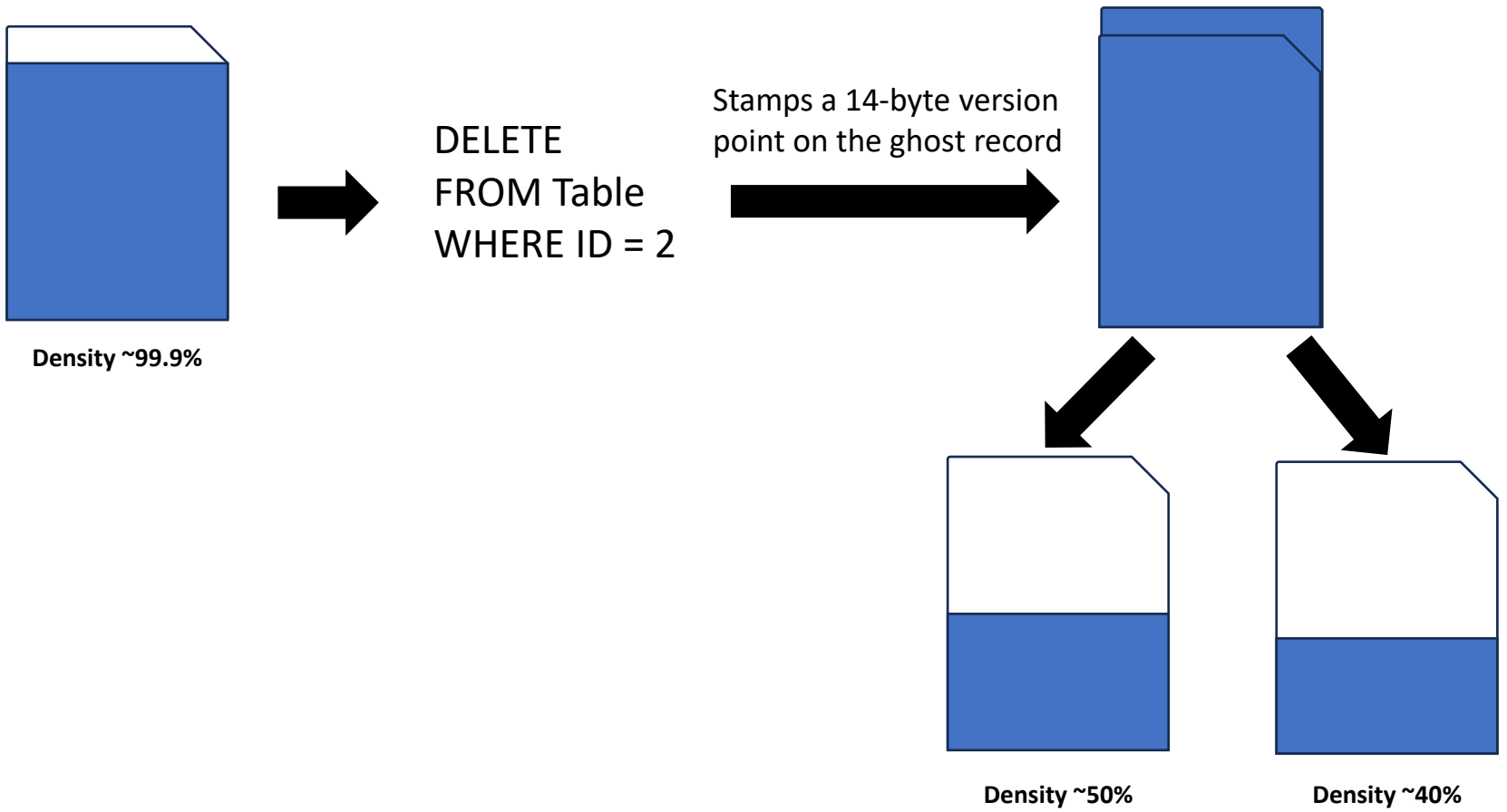
Don’t be surprised if the index size is bigger



Don’t be surprised if the index size is bigger

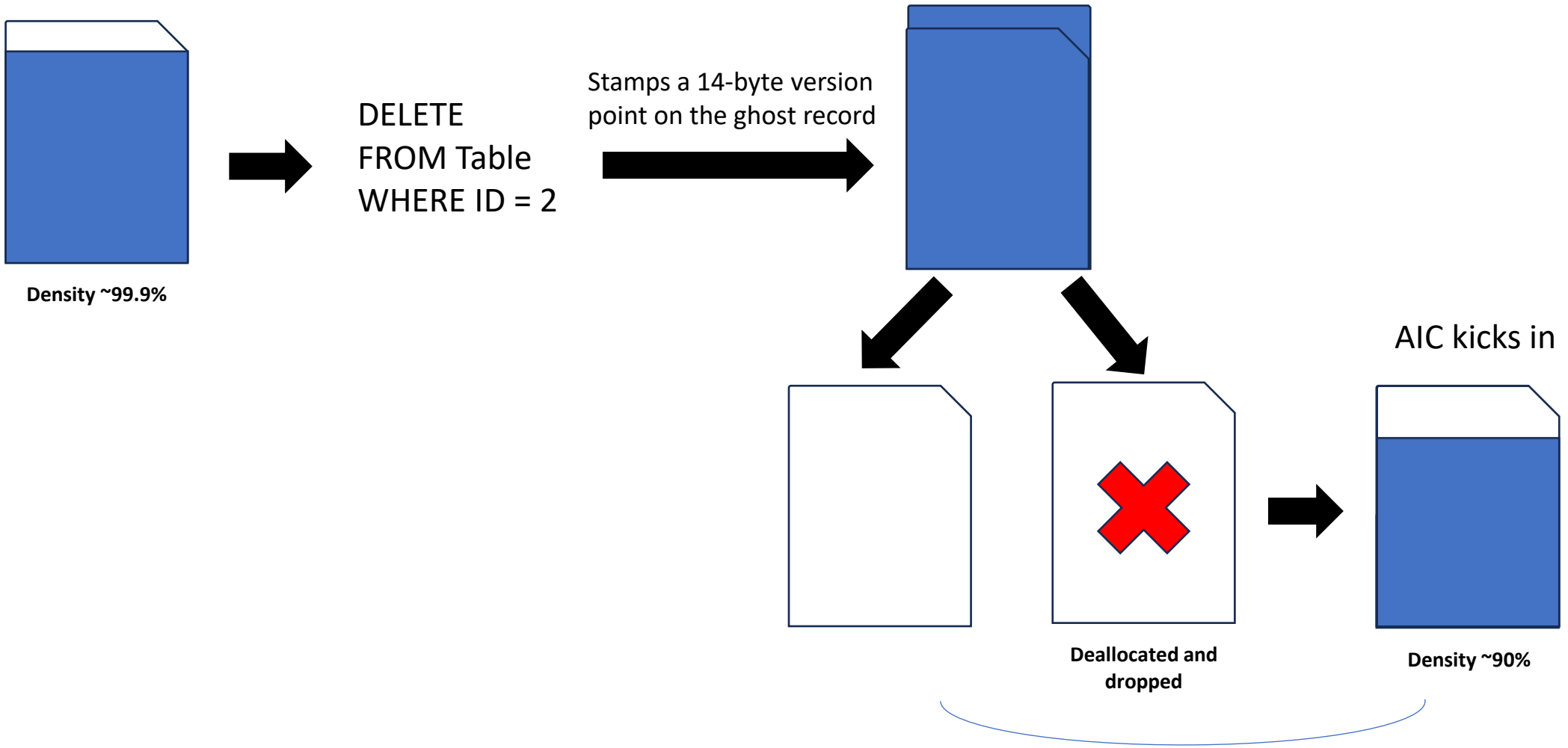


Don’t be surprised if the index size is bigger

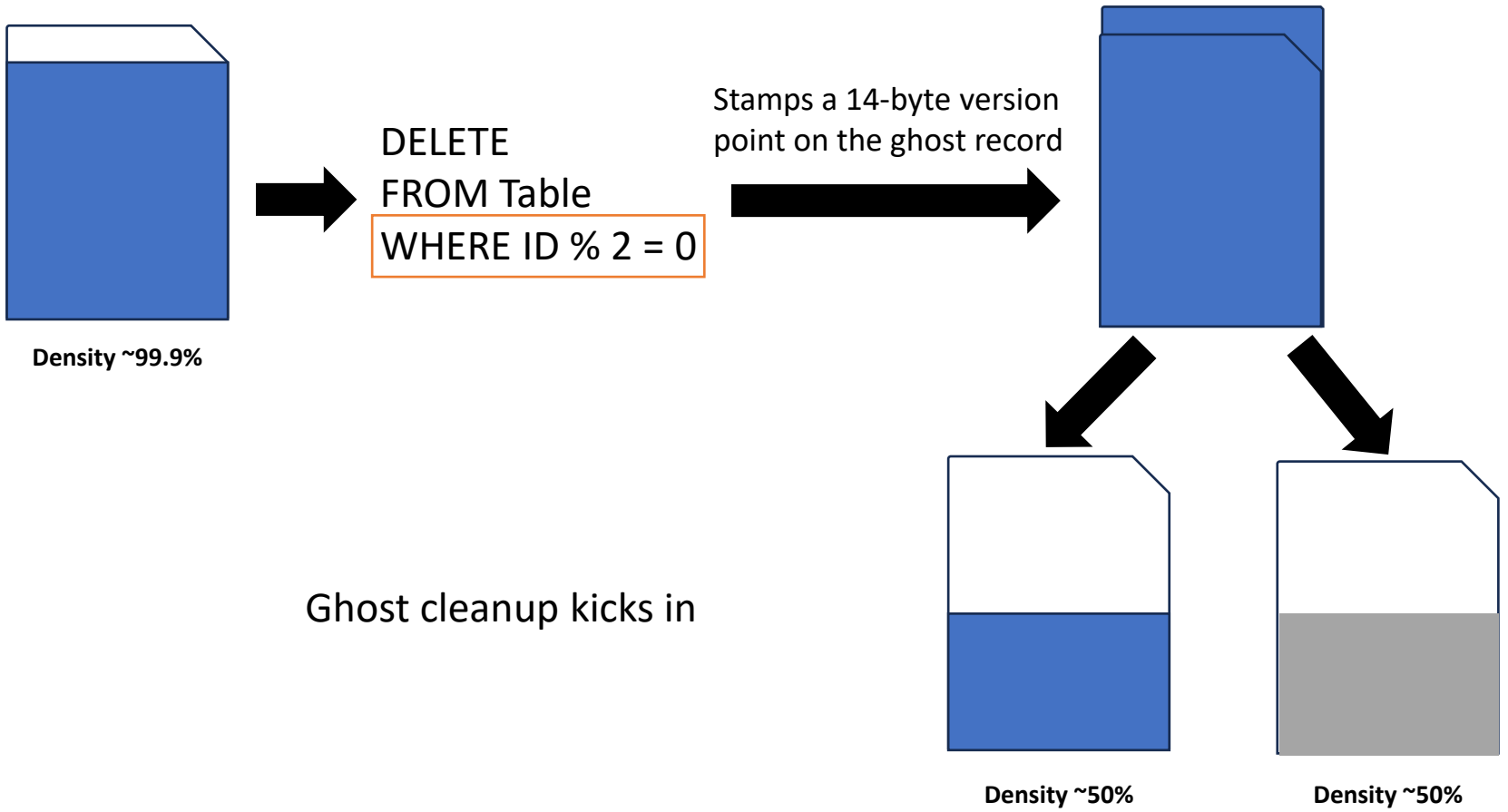


AIC kicks in

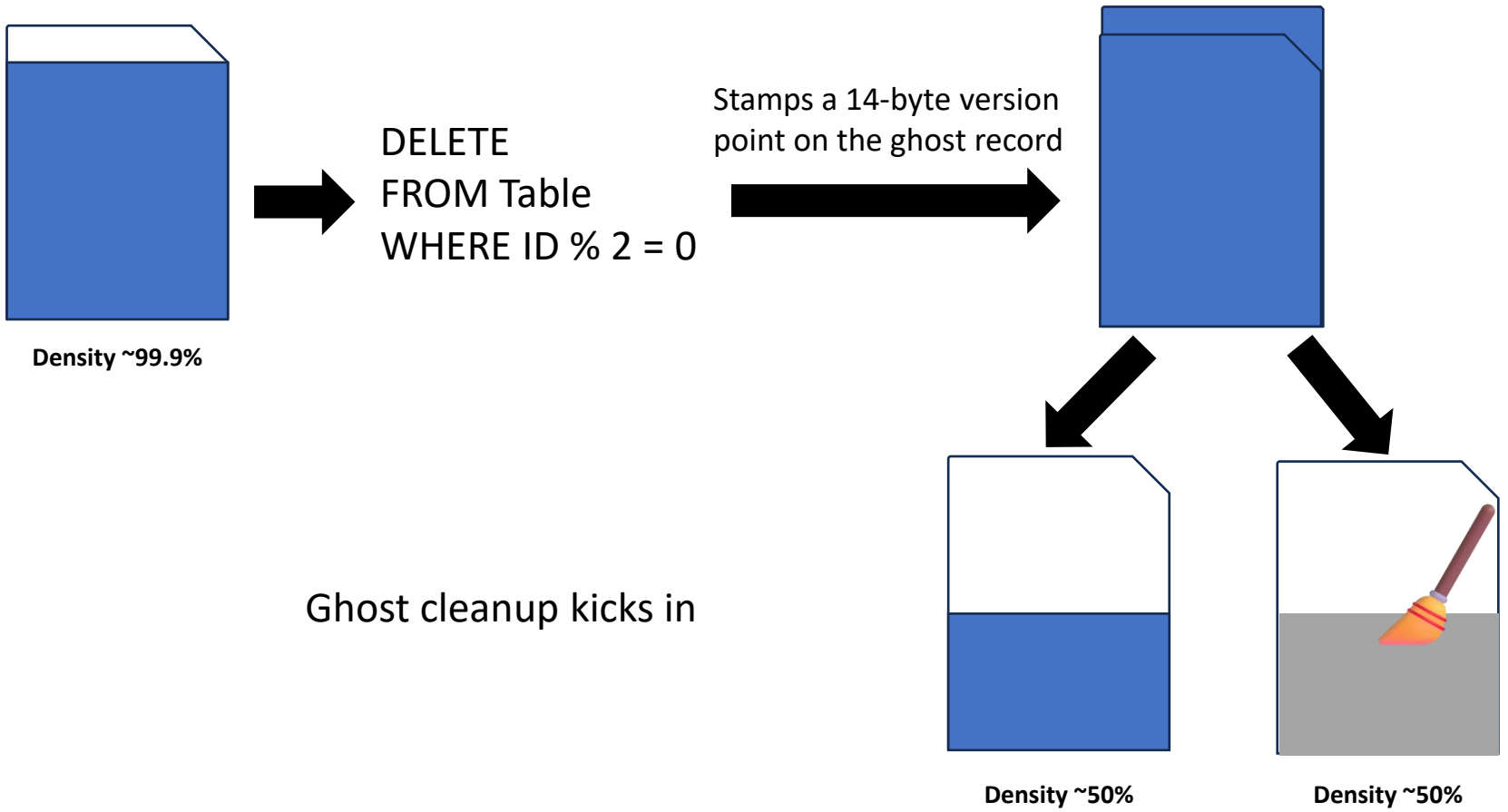
Don’t be surprised if the index size is bigger



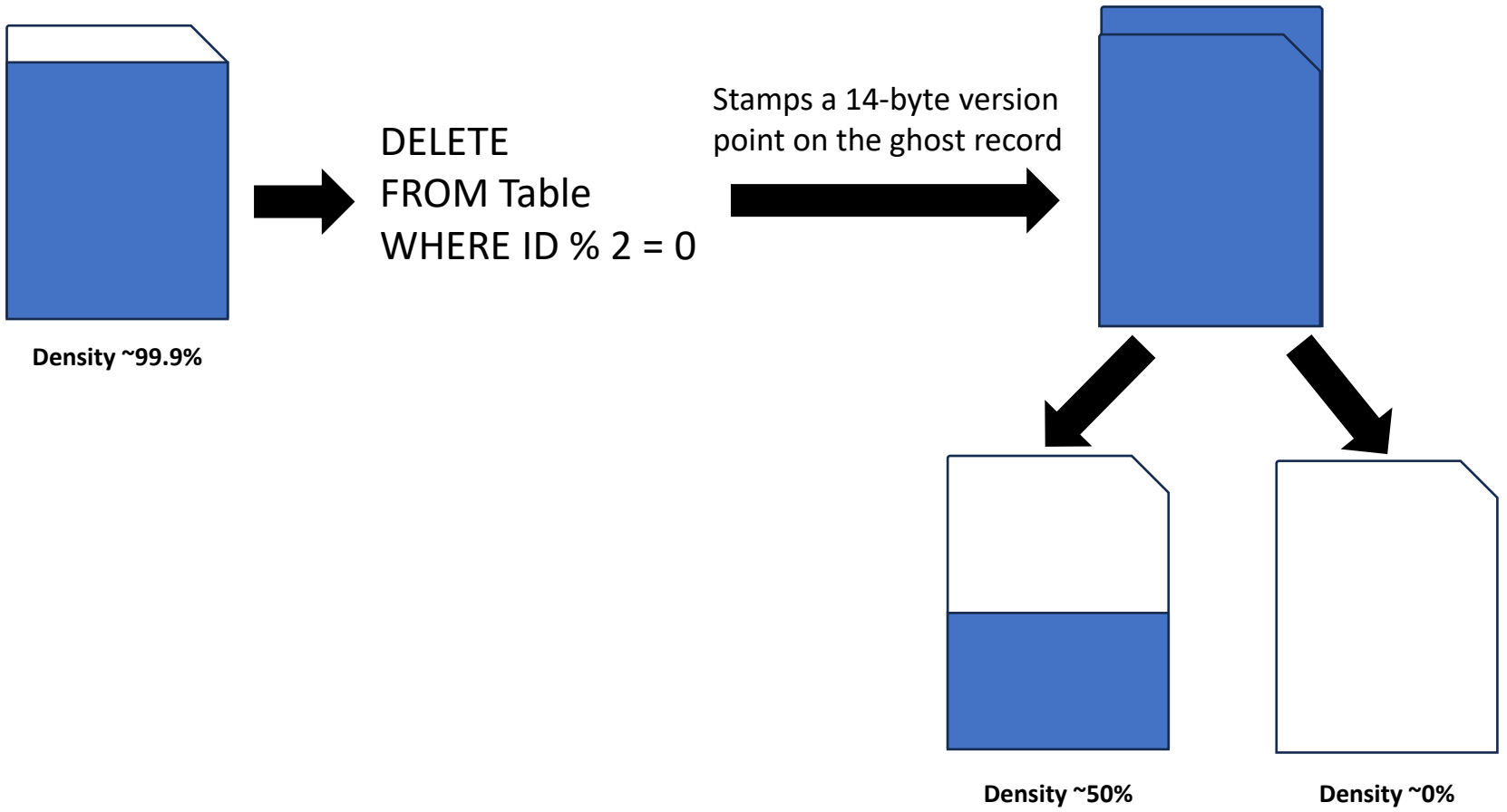
Don’t be surprised if the index size is bigger



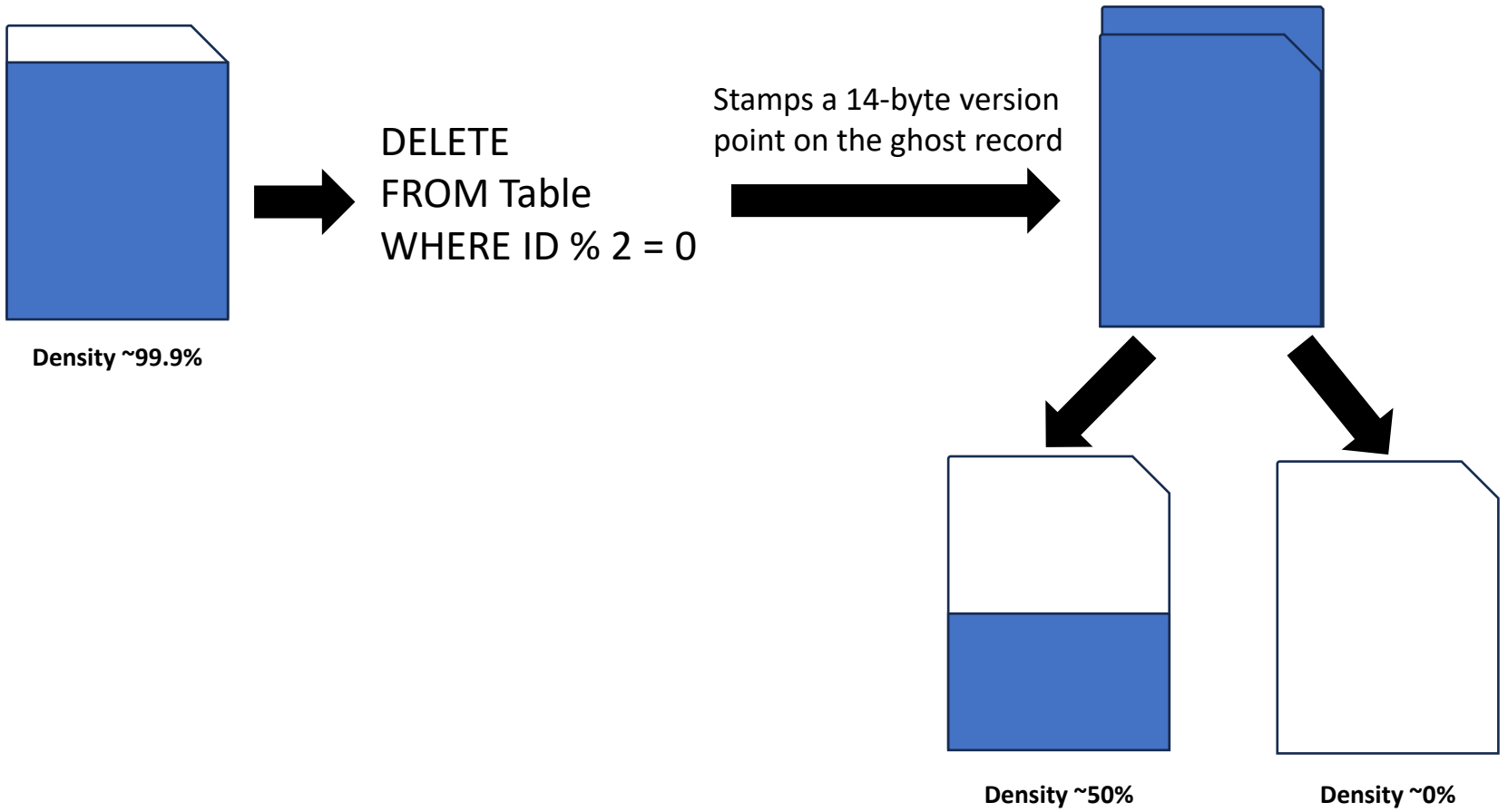
Don’t be surprised if the index size is bigger



Don’t be surprised if the index size is bigger

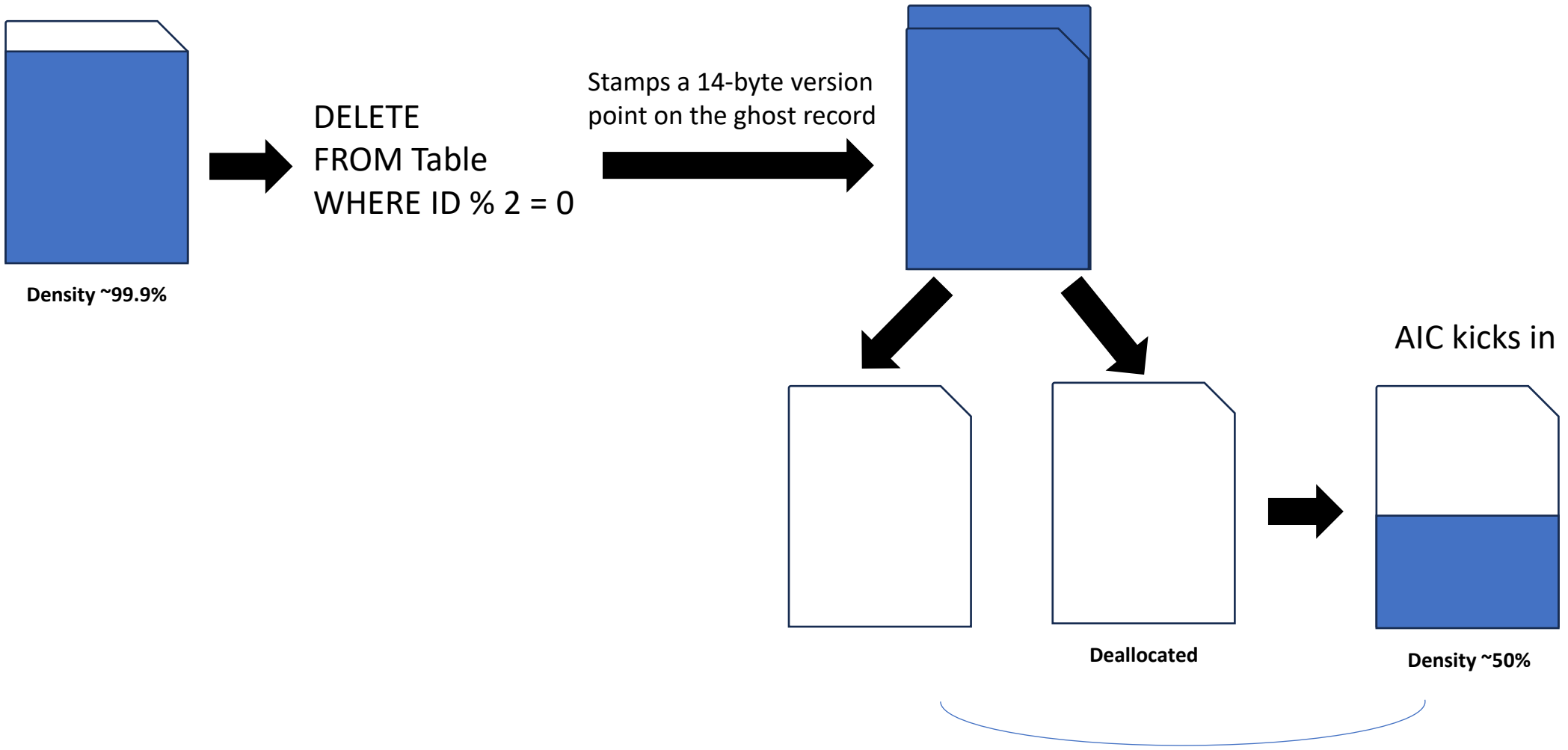


Don’t be surprised if the index size is bigger

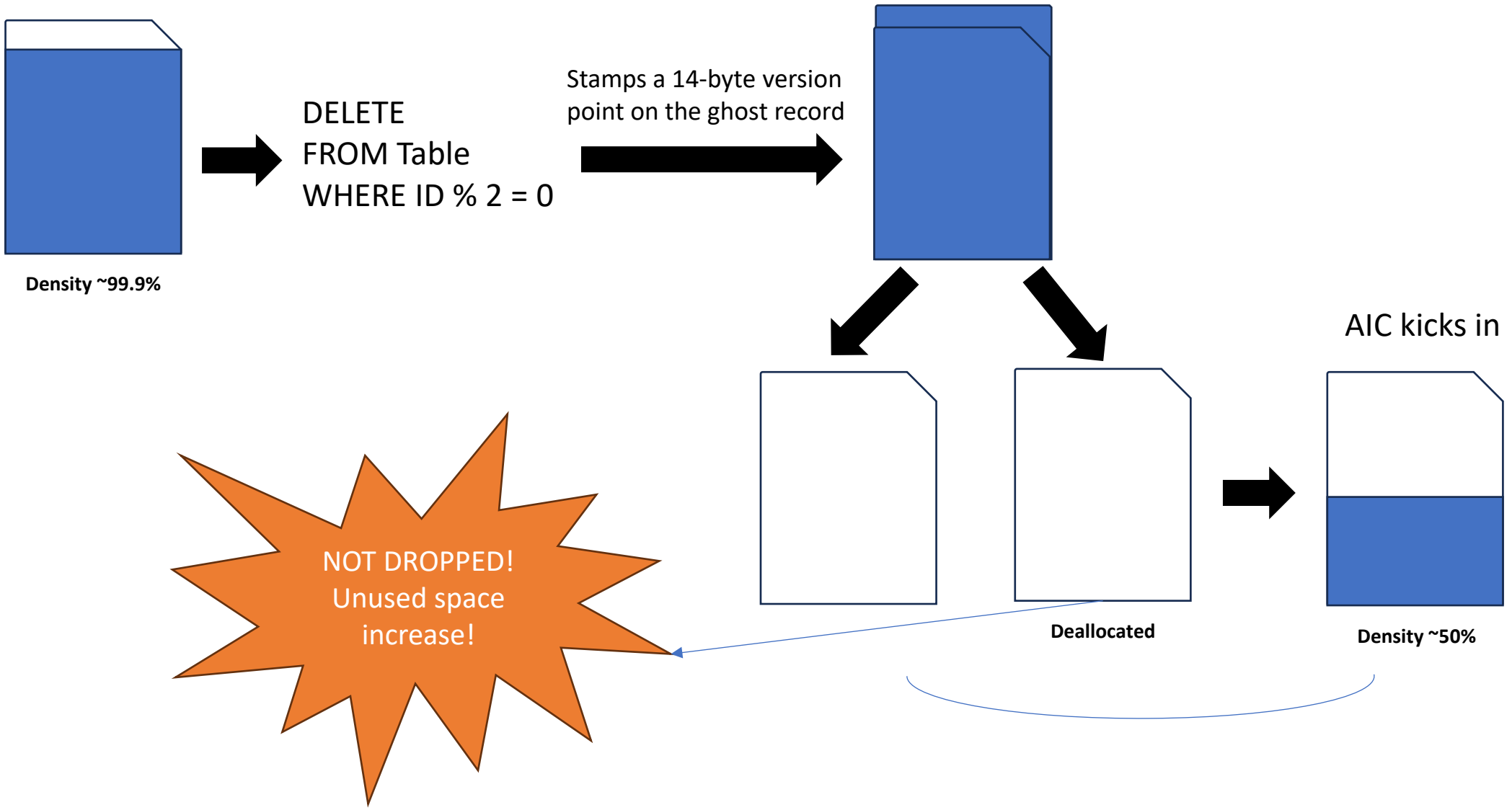


AIC kicks in

Don’t be surprised if the index size is bigger



Don’t be surprised if the index size is bigger





DEMO

What is the current status of AIC?



It is still in Preview!

What is the status?



~~It is still in Preview!~~



It was! Since two weeks ago, it is now a GA! (Announced at SQLCon)

However… for now…

It’s only available in *Azure SQL Database, Azure SQL Managed Instance with the "Always-up-to-date" update policy, and Fabric SQL DB.*

- Meaning we don’t have it on boxed SQL Server 2025 product.

Requirements



- Accelerated Database Recovery (ADR)
  - It's on by default in Azure SQL DB
  - It relies on the Persisted Version Store (PVS)

- It needs to be turned on
  - It's a **per-database** setting. (Would be better being per index?)
  - Use:

```
ALTER DATABASE CURRENT
SET AUTOMATIC_INDEX_COMPACTION = ON;
GO
```

- Doesn't need any restart or exclusive lock!

Limitations



- Only for clustered / nonclustered indexes
  - Not for HEAPS

- Only compacts leaf-level pages of the B-tree indexes in `IN_ROW_DATA` allocation units.
  - It won’t touch:
    - ROW_OVERFLOW / LOB_DATA pages.
    - Compressed columnstore rowgroups,
    - Memory-optimized tables
    - System tables
    - Indexes with page locks disabled (ALLOW_PAGE_LOCKS = OFF)

- Can be skipped if
  - PVS size >= 150GB
  - >= 1000 aborted transactions need cleanup
  - Pages are locked by an active transaction, concurrent rebuild/reorganize or shrink.

Comparison



Aspect	Traditional Index maintenance	Automatic Index Compaction
Trigger	Manual/scheduled rebuild or reorganize jobs	Continuous background process via PVS cleaner

Comparison

Aspect	Traditional Index maintenance	Automatic Index Compaction
Trigger	Manual/scheduled rebuild or reorganize jobs	Continuous background process via PVS cleaner
Scope scanned	Processes all pages in the index	Only recently modified pages

Comparison

Aspect	Traditional Index maintenance	Automatic Index Compaction
Trigger	Manual/scheduled rebuild or reorganize jobs	Continuous background process via PVS cleaner
Scope scanned	Processes all pages in the index	Only recently modified pages
Resource cost	High CPU, memory, log I/O in bursts	Low, spread continuously over time

Comparison

Aspect	Traditional Index maintenance	Automatic Index Compaction
Trigger	Manual/scheduled rebuild or reorganize jobs	Continuous background process via PVS cleaner
Scope scanned	Processes all pages in the index	Only recently modified pages
Resource cost	High CPU, memory, log I/O in bursts	Low, spread continuously over time
Free space needed	Rebuild needs space (at least equal to the index size)	No extra free space required

Comparison

Aspect	Traditional Index maintenance	Automatic Index Compaction
Trigger	Manual/scheduled rebuild or reorganize jobs	Continuous background process via PVS cleaner
Scope scanned	Processes all pages in the index	Only recently modified pages
Resource cost	High CPU, memory, log I/O in bursts	Low, spread continuously over time
Free space needed	Rebuild needs space (at least equal to the index size)	No extra free space required
Page density	Improves only after each maintenance run	Kept consistently high automatically

Comparison

Aspect	Traditional Index maintenance	Automatic Index Compaction
Trigger	Manual/scheduled rebuild or reorganize jobs	Continuous background process via PVS cleaner
Scope scanned	Processes all pages in the index	Only recently modified pages
Resource cost	High CPU, memory, log I/O in bursts	Low, spread continuously over time
Free space needed	Rebuild needs space (at least equal to the index size)	No extra free space required
Page density	Improves only after each maintenance run	Kept consistently high automatically
External Fragmentation	Reduced by rebuild/reorganize	Not addressed; can slightly increase

Comparison

Aspect	Traditional Index maintenance	Automatic Index Compaction
Trigger	Manual/scheduled rebuild or reorganize jobs	Continuous background process via PVS cleaner
Scope scanned	Processes all pages in the index	Only recently modified pages
Resource cost	High CPU, memory, log I/O in bursts	Low, spread continuously over time
Free space needed	Rebuild needs space (at least equal to the index size)	No extra free space required
Page density	Improves only after each maintenance run	Kept consistently high automatically
External Fragmentation	Reduced by rebuild/reorganize	Not addressed; can slightly increase
Statistics	Rebuild refreshes stats via full scan	Not updated; still needs separate job

Comparison

Aspect	Traditional Index maintenance	Automatic Index Compaction
Trigger	Manual/scheduled rebuild or reorganize jobs	Continuous background process via PVS cleaner
Scope scanned	Processes all pages in the index	Only recently modified pages
Resource cost	High CPU, memory, log I/O in bursts	Low, spread continuously over time
Free space needed	Rebuild needs space (at least equal to the index size)	No extra free space required
Page density	Improves only after each maintenance run	Kept consistently high automatically
External Fragmentation	Reduced by rebuild/reorganize	Not addressed; can slightly increase
Statistics	Rebuild refreshes stats via full scan	Not updated; still needs separate job
Azure resource impact	Can starve workload, delay replicas, HA risk	Minimal (low single-digit CPU overhead)

Comparison

Aspect	Traditional Index maintenance	Automatic Index Compaction
Trigger	Manual/scheduled rebuild or reorganize jobs	Continuous background process via PVS cleaner
Scope scanned	Processes all pages in the index	Only recently modified pages
Resource cost	High CPU, memory, log I/O in bursts	Low, spread continuously over time
Free space needed	Rebuild needs space (at least equal to the index size)	No extra free space required
Page density	Improves only after each maintenance run	Kept consistently high automatically
External Fragmentation	Reduced by rebuild/reorganize	Not addressed; can slightly increase
Statistics	Rebuild refreshes stats via full scan	Not updated; still needs separate job
Azure resource impact	Can starve workload, delay replicas, HA risk	Minimal (low single-digit CPU overhead)
Blocking	Rebuild/reorganize can hold locks longer	Short-term millisecond page locks only

Comparison

Aspect	Traditional Index maintenance	Automatic Index Compaction
Trigger	Manual/scheduled rebuild or reorganize jobs	Continuous background process via PVS cleaner
Scope scanned	Processes all pages in the index	Only recently modified pages
Resource cost	High CPU, memory, log I/O in bursts	Low, spread continuously over time
Free space needed	Rebuild needs space (at least equal to the index size)	No extra free space required
Page density	Improves only after each maintenance run	Kept consistently high automatically
External Fragmentation	Reduced by rebuild/reorganize	Not addressed; can slightly increase
Statistics	Rebuild refreshes stats via full scan	Not updated; still needs separate job
Azure resource impact	Can starve workload, delay replicas, HA risk	Minimal (low single-digit CPU overhead)
Blocking	Rebuild/reorganize can hold locks longer	Short-term millisecond page locks only
Storage growth	Grows until next maintenance window	Growth reduced continuously, in real time

Comparison

Aspect	Traditional Index maintenance	Automatic Index Compaction
Trigger	Manual/scheduled rebuild or reorganize jobs	Continuous background process via PVS cleaner
Scope scanned	Processes all pages in the index	Only recently modified pages
Resource cost	High CPU, memory, log I/O in bursts	Low, spread continuously over time
Free space needed	Rebuild needs space (at least equal to the index size)	No extra free space required
Page density	Improves only after each maintenance run	Kept consistently high automatically
External Fragmentation	Reduced by rebuild/reorganize	Not addressed; can slightly increase
Statistics	Rebuild refreshes stats via full scan	Not updated; still needs separate job
Azure resource impact	Can starve workload, delay replicas, HA risk	Minimal (low single-digit CPU overhead)
Blocking	Rebuild/reorganize can hold locks longer	Short-term millisecond page locks only
Storage growth	Grows until next maintenance window	Growth reduced continuously, in real time
File/allocated size	Rebuild can increase allocated data file space	Doesn't shrink allocated file size either

Comparison

Aspect	Traditional Index maintenance	Automatic Index Compaction
Trigger	Manual/scheduled rebuild or reorganize jobs	Continuous background process via PVS cleaner
Scope scanned	Processes all pages in the index	Only recently modified pages
Resource cost	High CPU, memory, log I/O in bursts	Low, spread continuously over time
Free space needed	Rebuild needs space (at least equal to the index size)	No extra free space required
Page density	Improves only after each maintenance run	Kept consistently high automatically
External Fragmentation	Reduced by rebuild/reorganize	Not addressed; can slightly increase
Statistics	Rebuild refreshes stats via full scan	Not updated; still needs separate job
Azure resource impact	Can starve workload, delay replicas, HA risk	Minimal (low single-digit CPU overhead)
Blocking	Rebuild/reorganize can hold locks longer	Short-term millisecond page locks only
Storage growth	Grows until next maintenance window	Growth reduced continuously, in real time
File/allocated size	Rebuild can increase allocated data file space	Doesn't shrink allocated file size either
Administration effort	Requires setup, scheduling, monitoring jobs	No jobs to create or maintain

Comparison

Aspect	Traditional Index maintenance	Automatic Index Compaction
Trigger	Manual/scheduled rebuild or reorganize jobs	Continuous background process via PVS cleaner
Scope scanned	Processes all pages in the index	Only recently modified pages
Resource cost	High CPU, memory, log I/O in bursts	Low, spread continuously over time
Free space needed	Rebuild needs space (at least equal to the index size)	No extra free space required
Page density	Improves only after each maintenance run	Kept consistently high automatically
External Fragmentation	Reduced by rebuild/reorganize	Not addressed; can slightly increase
Statistics	Rebuild refreshes stats via full scan	Not updated; still needs separate job
Azure resource impact	Can starve workload, delay replicas, HA risk	Minimal (low single-digit CPU overhead)
Blocking	Rebuild/reorganize can hold locks longer	Short-term millisecond page locks only
Storage growth	Grows until next maintenance window	Growth reduced continuously, in real time
File/allocated size	Rebuild can increase allocated data file space	Doesn't shrink allocated file size either
Administration effort	Requires setup, scheduling, monitoring jobs	No jobs to create or maintain
Index Size (# of pages)	Reduced but at higher cost	Reduced with less cost but not 100%

What do you think?



Is this the end of Index Maintenance?

Session Feedback 💖



Is this the end of Index Maintenance? Introducing Automatic Index Compaction